

Sistema de *leads* inteligente

¿Qué es un sistema de *leads* inteligente?

Un **Sistema de *Leads* Inteligente** es una infraestructura automatizada que captura, procesa, enriquece, califica y gestiona prospectos potenciales de manera sistemática. A diferencia de los sistemas tradicionales donde un equipo humano realiza estas tareas manualmente, un sistema inteligente utiliza automatización, APIs externas e inteligencia artificial para:

1. **Capturar *leads*** de múltiples fuentes (formularios web, LinkedIn, campañas publicitarias).
2. **Normalizar y validar** la información recibida.
3. **Enriquecer** los datos con información adicional de fuentes públicas y privadas.
4. **Calificar automáticamente** cada *lead* según su probabilidad de conversión.
5. **Distribuir** los *leads* al equipo de ventas apropiado.
6. **Automatizar** comunicaciones de seguimiento personalizadas.

Impacto en el negocio

Según estudios de 2025, las empresas que implementan sistemas de *leads* inteligentes experimentan:

- 70% de reducción en tiempo de respuesta a *leads*.
- 45% de mejora en tasas de conversión.
- 60% de disminución en costos operativos de ventas.
- 3x aumento en productividad del equipo comercial.



Pipeline de leads

¿Qué es un *pipeline de leads*?

Un ***pipeline de leads*** es el flujo estructurado que sigue un prospecto desde el momento en que es capturado hasta que se convierte en cliente o es descartado.

Piensa en él como una línea de ensamblaje donde cada estación agrega valor o toma decisiones sobre el lead.

Etapas típicas de un *pipeline*



Diseño de *pipelines*: ingesta y normalización de datos

Fuentes comunes de *leads*

1. Formularios Web

- Formularios en *landing pages*
- *Pop-ups* de suscripción
- *Chatbots* en sitio web

2. Publicidad digital

- Facebook *Lead Ads*
- Google *Ads Lead Forms*
- LinkedIn *Lead Gen Forms*

3. Redes Sociales

- Mensajes directos en Instagram
- Comentarios en publicaciones
- LinkedIn *Sales Navigator*

4. Eventos y *Webinars*

- Registros de asistentes
- Participantes de eventos virtuales

Desafío: datos heterogéneos

Cada fuente de *leads* proporciona datos en formatos diferentes:

Ejemplo de Facebook *Lead Ads*

```
{
  "field_data": [
    {"name": "full_name", "values": ["Juan Pérez García"]},
    {"name": "email", "values": ["juan.perez@empresa.com"]},
    {"name": "phone_number", "values": ["+34612345678"]},
    {"name": "company_name", "values": ["Tech Solutions SL"]}
  ]
}
```

Ejemplo en Google Forms

```
{  
  "responses": {  
    "Nombre completo": "Juan Pérez García",  
    "Correo electrónico": "juan.perez@empresa.com",  
    "Teléfono": "612 34 56 78",  
    "Empresa": "Tech Solutions SL"  
  }  
}
```

Ejemplo de formulario personalizado:

```
{  
  "contact": {  
    "firstName": "Juan",  
    "lastName": "Pérez García",  
    "email": "juan.perez@empresa.com",  
    "mobile": "612345678",  
    "companyName": "Tech Solutions SL"  
  }  
}
```

Normalización de Datos en n8n

Objetivo: Convertir todos estos formatos heterogéneos en una estructura única y consistente.



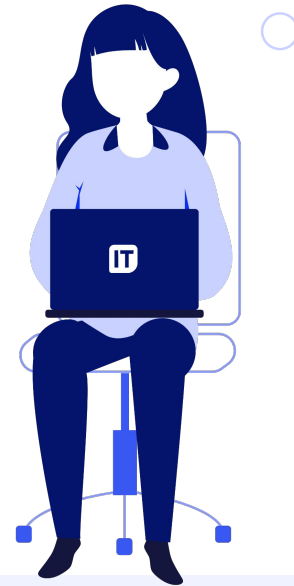
Estructura objetivo (normalizada)

```
{
  "lead": {
    "firstName": "Juan",
    "lastName": "Pérez García",
    "fullName": "Juan Pérez García",
    "email": "juan.perez@empresa.com",
    "phone": "+34612345678",
    "company": "Tech Solutions SL",
    "source": "facebook_ads",
    "capturedAt": "2025-10-15T10:30:00Z",
    "status": "new"
  }
}
```


Workflow en n8n para normalización

1. **Nodo Webhook:** recibe el *lead* de cualquier fuente.
2. **Nodo Switch:** identifica la fuente según parámetros o estructura.
3. **Nodos Function:** uno por cada fuente, normaliza el formato.
4. **Nodo Set:** crea la estructura final unificada.

Ejemplo de decisión inteligente: si el campo “mensaje” indica urgencia, el sistema puede marcar el *lead* como prioritario incluso antes del scoring completo.



Código de ejemplo para *nodo Function* (Facebook Ads):

```
// Normalización de lead de Facebook Ads

const facebookData = $input.item.json.field_data;

// Función auxiliar para extraer valores
function getFieldValue(fieldData, fieldName) {
  const field = fieldData.find(f => f.name === fieldName);
  return field ? field.values[0] : null;
}

// Extraer y normalizar datos
const fullName = getFieldValue(facebookData, 'full_name') || '';
const nameParts = fullName.trim().split(/\s+/);
const firstName = nameParts[0] || '';
const lastName = nameParts.slice(1).join(' ') || '';
```

...

```
// Normalizar teléfono (agregar código de país si falta)
let phone = getFieldValue facebookData, 'phone_number' || '';
if (phone) {
  phone = phone.replace(/\s/g, '');
  if (!phone.startsWith('+')) {
    phone = '+34' + phone;
  }
}

// Crear estructura normalizada
return {
  json: {
    lead: {
      firstName,
      lastName,
      fullName,
      email: (getFieldValue facebookData, 'email' || '').toLowerCase().trim(),
      phone,
      company: getFieldValue facebookData, 'company_name' || '',
      source: 'facebook_ads',
      capturedAt: new Date().toISOString(),
      status: 'new',
      rawData: $input.item.json // Guardar datos originales
    }
  }
};
```

Validación y limpieza de datos con nodos de n8n

La validación asegura que los datos recibidos sean utilizables y de calidad. Un *lead* con un *email* inválido o un teléfono incorrecto genera pérdida de tiempo y recursos.

Por eso, antes de enriquecer o calificar un *lead*, el *workflow* debe validar y limpiar la información básica.



Validaciones Esenciales

Validación de *Email*

Nodo Function: validación básica:



} Veamos el código
en la siguiente *slide*.

```
// Obtener el email desde el JSON de entrada
const email = $json.lead.email || '';

// Expresión regular para validación básica de formato de email
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const isValidFormat = emailRegex.test(email);

// Verificar dominios temporales o desechables
const disposableDomains = ['tempmail.com', 'guerrillamail.com', '10minutemail.com'];
const domain = email.split('@')[1]?.toLowerCase() || '';
const isDisposable = disposableDomains.includes(domain);

// Validaciones adicionales (placeholder para verificación MX)
const hasValidMX = true; // Implementar verificación MX si es necesario
```

...

```
// Retornar resultado de validación
return {
  json: {
    ...$json,
    validation: {
      email: {
        isValid: isValidFormat && !isDisposable && hasValidMX,
        isFormat: isValidFormat,
        isDisposable,
        domain
      }
    }
  }
};
```

Si el *email* es temporal o inválido, el *lead* puede marcarse para revisión. Si falta el dominio, el sistema no debería intentar buscar datos de empresa.

Validación de teléfono

```
// Obtener el teléfono desde el JSON de entrada
const phone = $json.lead.phone || '';

// Normalizar teléfono: eliminar espacios, guiones y paréntesis
const normalizedPhone = phone.replace(/[\s\-\(\)]/g, '');

// Validar formato internacional (E.164)
const phoneRegex = /^+?[1-9]\d{7,14}$/;
const isValidFormat = phoneRegex.test(normalizedPhone);

// Detectar código de país (si existe)
const match = normalizedPhone.match(/^+(\d{1,3})/);
const countryCode = match ? match[1] : null;
```

...

```
// Retornar estructura con validación y normalización
return {
  json: {
    ...$json,
    lead: {
      ...$json.lead,
      phone: normalizedPhone
    },
    validation: {
      ...$json.validation,
      phone: {
        isValid: isValidFormat,
        normalized: normalizedPhone,
        countryCode
      }
    }
  }
};
```


Limpieza de nombre de empresa

```
// Obtener el nombre de la empresa desde el JSON de entrada
const company = $json.lead.company || '';

// Función para capitalizar la primera letra de cada palabra
function titleCase(str) {
  return str
    .toLowerCase()
    .replace(/\b\w/g, l => l.toUpperCase());
}

// Eliminar sufijos legales comunes (S.L., S.A., Inc., etc.)
const legalSuffixes = ['S.L.', 'SL', 'S.A.', 'SA', 'Inc.', 'LLC', 'Ltd.'];

let cleanCompany = company;

legalSuffixes.forEach(suffix => {
  const regex = new RegExp(`\\s*${suffix}\\s*$`, 'i');
  cleanCompany = cleanCompany.replace(regex, '');
});
```

...

```
// Normalizar: eliminar espacios y aplicar formato título
cleanCompany = titleCase(cleanCompany.trim());

// Retornar resultado normalizado
return {
  json: {
    ...$json,
    lead: {
      ...$json.lead,
      company: cleanCompany,
      companyOriginal: company
    }
  }
};
```

Deduplicación de *leads*

Antes de procesar un *lead*, verifica si ya existe en tu sistema.

```
// Buscar lead existente por email
const email = $json.lead.email || '';

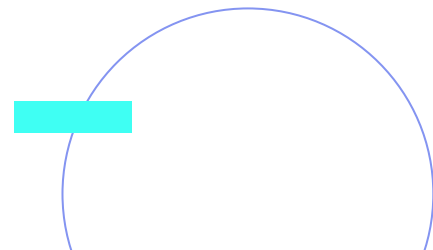
// Resultado de búsqueda en CRM (ejemplo)
const existingLead = $json.existingLeadData || [];

// Evaluar si existe un lead duplicado
if (Array.isArray(existingLead) && existingLead.length > 0) {
  // Lead duplicado encontrado
  return {
    json: {
      ...$json,
      isDuplicate: true,
      duplicateId: existingLead[0].id || null,
      action: 'update' // o 'skip' según las reglas de negocio
    }
  };
}
```

...

```
} else {  
  // Lead nuevo  
  return {  
    json: {  
      ...$json,  
      isDuplicate: false,  
      action: 'create'  
    }  
  };  
}
```

La deduplicación permite decidir si corresponde crear un nuevo *lead*, actualizar un *lead* existente, fusionar información nueva con datos previos, evitar una comunicación duplicada o escalar el *lead* si mostró interés por más de un canal.



Endpoints y APIs

¿Qué es un *Endpoint* ?

Un **endpoint** es una URL específica dentro de una API donde tu aplicación (en este caso, n8n) puede recibir o enviar datos. En el contexto de *leads*, un *endpoint* típico es una URL de *webhook* que recibe datos de formularios.

Ejemplo de endpoint en n8n

<https://tu-instancia.n8n.cloud/webhook/captura-leads-facebook>

Cuando otro sistema envía datos a esa URL, n8n puede iniciar el *workflow*.



Creación de *Endpoints* en n8n

Paso 1: Configurar *Webhook Node*

1. Agrega un nodo **Webhook** a tu *workflow*
2. Configura los parámetros:
 - **HTTP Method:** POST (para recibir datos).
 - **Path:** captura-leads-facebook.
 - **Authentication:** *Basic Auth* (opcional, recomendado).
 - **Response Mode:** "When Last Node Finishes".
 - **Response Code:** 200.

Paso 2: Definir estructura de respuesta

Agrega un nodo **Respond to Webhook** al final del *workflow*.

```
{
  "success": true,
  "message": "Lead recibido correctamente",
  "leadId": "{{ $json.lead.id }}",
  "timestamp": "{{ $now.toISO() }}"
}
```

Webhooks : configuración y mejores prácticas

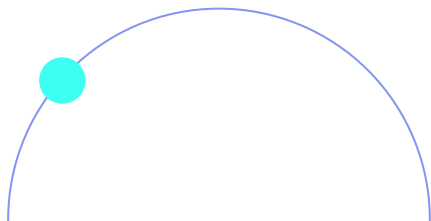
¿Qué es un Webhook?

Un **webhook** es un mecanismo de comunicación HTTP que permite a una aplicación enviar datos en tiempo real a otra aplicación cuando ocurre un evento específico.

Dicho simple: es como un “timbre digital”. Cuando pasa un evento, una plataforma envía datos a una URL específica para activar una acción en otro sistema.

Diferencia clave:

- **API tradicional:** Tu aplicación pregunta "¿hay algo nuevo?" (*polling*).
- **Webhook:** La aplicación externa te dice "aquí hay algo nuevo" (*push*).



Configuración de *Webhooks* en n8n

Modo Test vs. Production

n8n proporciona dos URLs de *webhook*:

- **Test URL:** Solo funciona cuando el *workflow* está en modo "*listening*".

<https://tu-instancia.n8n.cloud/webhook-test/captura-leads>

- **Production URL:** Funciona siempre que el *workflow* esté activado.

<https://tu-instancia.n8n.cloud/webhook/captura-leads>

Importante: Usa la URL de producción en integraciones reales.



Seguridad de Webhooks

Autenticación básica

Configura credenciales en el nodo *Webhook*:

- *Username*:

```
api_user_leads
```

- *Password*:

```
[genera-contraseña-segura]
```

La URL a proporcionar será:

```
https://api_user_leads:[password]@tu-instancia  
.n8n.cloud/webhook/captura-leads
```

Validación de Firma (*Signature Verification*)

Muchas plataformas (Facebook, Stripe, etc.) envían una firma criptográfica para verificar autenticidad.



} Veamos el código de la siguiente pantalla.

```
// Nodo Function - Validación de firma de Facebook

const crypto = require("crypto");

// Extraer encabezado y cuerpo del webhook
const signature = $node["Webhook"].json.headers["x-hub-signature-256"];
const payload = JSON.stringify($node["Webhook"].json.body);
const appSecret = "tu-app-secret-de-facebook"; // ⚠ Reemplazá con tu App Secret real

// Calcular firma esperada
const expectedSignature =
  "sha256=" +
  crypto
    .createHmac("sha256", appSecret)
    .update(payload)
    .digest("hex");
```

...

```
// Verificar existencia de la firma antes de comparar
if (!signature) {
  throw new Error("Missing signature header (x-hub-signature-256)");
}

// Comparar firmas con seguridad frente a ataques de tiempo
const isValid = crypto.timingSafeEqual(
  Buffer.from(signature),
  Buffer.from(expectedSignature)
);

// Validar resultado
if (!isValid) {
  throw new Error("Invalid webhook signature – potential security threat");
}

// Si es válida, continuar flujo
return {
  json: {
    signatureValid: true,
    ...$json
  }
};
```

Validación de IP (*Whitelist*)

Para mayor seguridad, valida que los *requests* vengan solo de IPs conocidas:

```
// Obtener IP del request
const clientIP = $node["Webhook"].json.headers['x-forwarded-for'] ||
  $node["Webhook"].json.headers['x-real-ip'];

// IPs permitidas (ejemplo: IPs de Facebook)
const allowedIPs = [
  '173.252.88.0/24',
  '173.252.120.0/24',
  // ... más rangos
];

// Validar (implementación simplificada)
const isAllowedIP = allowedIPs.some(range => {
  // Implementar lógica de verificación de rango IP
  return clientIP.startsWith(range.split('/')[0].split('.').slice(0, 3).join('.'));
});
```

...

```
if (!isAllowedIP) {  
  throw new Error('Unauthorized IP address');  
}  
  
return {  
  json: {  
    ipValidated: true,  
    clientIP,  
    ...$json  
  }  
};
```



Manejo de errores y reintentos

Las plataformas externas esperan respuestas rápidas de los *webhooks* (típicamente < 5 segundos). Si tu *workflow* es complejo:



Estrategia: respuesta inmediata + procesamiento asíncrono

1. **Webhook Node:** Recibe datos.
2. **Respond to Webhook:** Responde inmediatamente con 200 OK.
3. **Queue/Database:** Guarda *lead* en cola de procesamiento.
4. **Workflow Separado:** Procesa *leads* de la cola.

Implementación con Google Sheets como cola

```
// Nodo Set - Preparar para inserción en "cola"
return {
  json: {
    leadId: Date.now().toString(),
    receivedAt: new Date().toISOString(),
    status: 'pending',
    rawData: JSON.stringify($json),
    retries: 0,
    processedAt: null
  }
};
```



Luego un *workflow* separado con ***Schedule Trigger***
(cada 1 minuto) procesa *leads* pendientes.

